

Decision Tree (Albero di Decisione)

Modello

Un albero di decisione è un modello che divide ricorsivamente lo spazio dei dati in regioni rettangolari, creando una **struttura gerarchica di nodi decisionali**.

Logica dell'Algoritmo

L'algoritmo costruisce l'albero utilizzando un approccio **greedy dall'alto verso il basso** (top-down):

1. Inizio con tutte le osservazioni nel nodo radice
2. Ad ogni nodo, scegliere la feature e il threshold che **massimizzano la purezza** (classificazione) o **minimizzano la varianza** (regressione)
3. Dividere ricorsivamente i nodi figli fino a criteri di stop (profondità massima, numero minimo di campioni, purezza perfetta)

Il risultato è una **struttura ad albero** dove:

- **Nodi interni:** contengono regole decisionali (feature <= threshold?)
- **Foglie:** contengono predizioni (classe o valore numerico)

Criteri di Split

Gini Index (Classificazione)

Misura la **purezza di un nodo** per la classificazione:

$$\text{Gini}(D) = 1 - \sum_{i=1}^K p_i^2$$

Dove p_i è la proporzione di istanze appartenenti alla classe i nel nodo.

- **Gini = 0:** nodo puro (tutte le istanze della stessa classe)
- **Gini = 1 - 1/K:** nodo completamente impuro (distribuito uniformemente)
- **Per K=2 (binario):** massimo = 0.5

Information Gain: al fare uno split, il guadagno è misurato come la riduzione di Gini media pesata:

$$\text{IG} = \text{Gini}(\text{padre}) - \sum_{\text{figlio}} \frac{|D_{\text{figlio}}|}{|D|} \text{Gini}(\text{figlio})$$

Entropia (Classificazione)

Misura l'**incertezza informativa** di un nodo:

$$\text{Entropia}(D) = - \sum_{i=1}^K p_i \log_2(p_i)$$

- **Entropia = 0:** nodo puro
- **Entropia = $\log_2(K)$:** nodo completamente impuro
- **Per K=2:** massimo = 1

Information Gain (con Entropia): il guadagno è la riduzione di entropia:

$$\text{IG} = \text{Entropia}(\text{padre}) - \sum_{\text{figlio}} \frac{|D_{\text{figlio}}|}{|D|} \text{Entropia}(\text{figlio})$$

Nota: Gini e Entropia producono risultati simili in pratica; Gini è computazionalmente più efficiente. L'Information Gain è usato nelle prime implementazioni di Decision Tree (CART).

Mean Squared Error (MSE) - Regressione

Misura la **varianza all'interno di un nodo** per problemi di regressione:

$$\text{MSE}(D) = \frac{1}{|D|} \sum_{i=1}^{|D|} (y_i - \bar{y})^2$$

Dove $\bar{y} = \frac{1}{|D|} \sum_{i=1}^{|D|} y_i$ è la media dei target nel nodo.

Lo split è scelto per **minimizzare la somma pesata di MSE** nei nodi figli.

Complessità Computazionale

Training (Costruzione dell'Albero)

La complessità dipende dalla **profondità massima** dell'albero e dal numero di feature:

Nel caso generale:

$$O(n \cdot p \cdot \log(n) \cdot d)$$

Dove:

- n = numero di osservazioni
- p = numero di feature
- $\log(n) \approx$ profondità dell'albero (se bilanciato)
- d = profondità effettiva ($\leq \log(n)$ per albero bilanciato)

Nella pratica:

- Per ogni nodo, scansionare tutte le p feature
- Per ogni feature, ordinare i dati (o usare binning per velocizzare) $\rightarrow O(n \log(n))$
- Questo accade ad ogni profondità dell'albero
- Se l'albero è bilanciato, profondità $\approx \log(n)$

Semplificazione:

Un albero completamente bilanciato ha complessità $O(n \cdot p \cdot \log^2(n))$

Nota importante: Un albero **completamente sviluppato** (no pruning) su n campioni può avere profondità fino a n (albero degenerato a lista), con complessità $O(n^2 \cdot p)$ — questo è un limite teorico, ma dimostra il rischio di overfitting.

Inference (Previsione)

$$O(d)$$

Dove d è la **profondità dell'albero**. Basta attraversare il cammino dalla radice alla foglia.

- Per alberi bilanciati: $O(\log(n))$
- Per alberi degenerati: $O(n)$

Memoria

$$O(n \cdot d + p)$$

- Memorizzazione dell'albero stesso: proporzionale al numero di nodi
- In un albero bilanciato: $O(n)$ nodi nel peggiore, $O(2^d) = O(n)$ nel medio
- Spazio per i dati di training: $O(n \cdot p)$

Considerazioni sulla Scalabilità

• Vantaggi:

- Training veloce per **piccoli-medi dataset**
- Inference molto veloce anche su dataset grandi
- Parallelizzabile a livello di feature (considerare split in parallelo per ogni feature)

• Svantaggi:

- Man mano che n cresce, il tempo quadratico $O(n^2)$ nel caso degenerato diventa problematico
- Per dataset enormi, tecniche di **gradient-based tree** (XGBoost, LightGBM) sono preferibili, che usano binning per ridurre $O(n \log n)$ a $O(n)$ per feature

Rappresentazione Interna

Un decision tree è rappresentato internamente come una **struttura ad albero ricorsiva**:

```
Nodo(feature=x1, threshold=5.5, left=Nodo(...), right=Nodo(...))
```

Ogni nodo contiene:

- **Condizione:** quale feature e quale threshold
- **Puntatori ai figli:** rami sinistro e destro
- **Informazioni di predizione:** classe modale (classificazione) o valore medio (regressione)

Implicazioni per la Spiegabilità

Vantaggi:

- La struttura è **completamente esplicita e navigabile**
- Ogni previsione ha un **tracciamento completo**: seguire il cammino dalla radice alla foglia spiega perfettamente come è stata raggiunta
- Non esiste ambiguità o "magia": è una sequenza di confronti

Svantaggi:

- **Alberi profondi o complessi** diventano difficili da interpretare (decine o centinaia di nodi)
- Le interazioni tra feature sono **implicite nella struttura**: se feature A e B interagiscono, l'albero le cattura tramite split in serie, ma questo non è ovvio dal grafico
- **Instabilità**: piccoli cambiamenti nei dati possono portare a strutture completamente diverse (vedi sezione Limiti)

Rispetto a modelli lineari (LR, logistica), la rappresentazione è **meno compatta ma più complessa**.

Vincoli sui Dati

Dataset Bilanciato

I decision tree possono essere **influenzati da classi sbilanciate**:

- L'algoritmo greedy tende a favorire split che massimizzano il puro della classe **maggioritaria**
- Su un dataset con 95% negativi e 5% positivi, un albero potrebbe diventare molto profondo per catturare i pochi positivi
- **Conseguenza**: scarsa capacità di classificazione sulla classe minoritaria (FN altissimi)

Soluzioni:

- Assegnare **pesi di classe** inversi alla frequenza
- Ricampionare (oversampling della classe rara, undersampling della maggioritaria)
- Usare metriche di valutazione appropriate (F1, non Accuracy)

Nessun Ulteriore Vincolo Strutturale

A differenza della regressione lineare e logistica, gli alberi di decisione **non hanno assunzioni** su:

- Linearità
- Normalità di distribuzioni
- Omoschedasticità
- Indipendenza di feature

Questo è un vantaggio (flessibilità), ma anche un rischio (overfitting senza controllo).

Capacità Predittive

Punti di Forza

- **Pattern non lineari:** cattura relazioni complesse e non lineari che modelli lineari non vedono
- **Interazioni automatiche:** le interazioni tra feature sono catturate naturalmente dalla struttura
- **Feature categoriche:** gestisce feature categoriche senza necessità di encoding
- **Nessuna assunzione:** nessun prerequisito su distribuzione o linearità dei dati

Punti di Debolezza

- **Pattern lineari:** su dati puramente lineari, gli alberi sono **inefficienti e imprecisi**
 - Avanzano tramite split ortogonali, creando funzioni a gradini
 - La predizione cambia bruscamente al passaggio di un threshold
 - Un modello lineare semplice avrebbe R^2 più alto con meno parametri
- **Dati sparsi ad alta dimensionalità:** con molte feature e pochi campioni, il rischio di overfitting è massimo

Metriche per la Confidenza

Metriche di Classificazione

Utilizza le stesse metriche della **Regressione Logistica**:

- **Confusion Matrix:** TP, TN, FP, FN
- **Accuracy:** $(TP + TN) / (TP + TN + FP + FN)$
- **Sensitivity / Recall:** $TP / (TP + FN)$
- **Specificity:** $TN / (TN + FP)$
- **Precision:** $TP / (TP + FP)$
- **F1-Score:** media armonica di Precision e Recall
- **ROC Curve e AUC:** trade-off tra TPR e FPR

Metriche di Regressione

Per alberi di regressione:

- **MSE:** errore quadratico medio
- **RMSE:** radice quadrata di MSE
- **MAE:** errore assoluto medio
- **R^2 :** frazione di varianza spiegata

Probabilità di Foglia (Confidence Score)

Un vantaggio specifico degli alberi è che forniscono naturalmente una **stima di confidenza della previsione**:

$$\text{Confidence} = \frac{\# \text{ istanze della classe predetta nella foglia}}{\# \text{ istanze totali nella foglia}}$$

Se una foglia contiene 100 campioni e 95 appartengono alla classe positiva, la confidenza della previsione "positivo" è 0.95.

Utilizzo: si può filtrare le predizioni per accuratezza: accettare solo previsioni con confidenza > 0.8.

Metriche per la Comprensione e Spiegabilità

Visualizzazione dell'Albero

Il principale strumento è la **visualizzazione grafica dell'albero**:

- Ogni nodo mostra la condizione di split e la distribuzione delle classi
- Foglie mostrano la previsione e il numero di campioni

Limite: su alberi profondi (>5-6 livelli), la visualizzazione diventa illeggibile

Feature Importance

Misura quante volte una feature viene utilizzata come criterio di split e quanto riduce l'impurità media:

$$\text{Importance}_j = \frac{\sum_{\text{nodi con feature } j} (\text{IG}_{\text{nodo}}) \times (\text{n campioni nodo})}{n}$$

somma su tutti i nodi

Interpretazione: feature con importanza alta sono state cruciali per le decisioni dell'albero.

Spiegazione Locale (Path to Prediction)

Per una singola istanza:

1. Tracciare il cammino dalla radice alla foglia
2. Elencare tutti i confronti (feature <= threshold) che hanno determinato la previsione

Esempio:

Istanza X predetta come "Sì" perché:

- Age <= 35 ✓
 - Income > 50000 ✓
 - CreditScore <= 700 ✓
- Foglia: Sì (95 istanze, 85 positive)

Questo è un **vantaggio enorme per la spiegabilità** rispetto a modelli "scatola nera". Ogni previsione è completamente tracciabile.

Limiti di Predizione

Overfitting

Il limite **principale** degli alberi di decisione. Senza controllo:

- L'albero continua a dividersi fino a quando ogni foglia è pura (una sola classe)
- Su dataset piccoli, ogni campione potrebbe trovarsi in una foglia propria
- **Risultato:** $R^2 = 1.0$ sul training set, ma pessimo su test set

Soluzioni:

- **Pruning:** rimuovere nodi che non migliorano significativamente la generalizzazione
- **Limiti sulla crescita:** profondità massima, numero minimo di campioni per split, impurità minima per split
- **Ensemble methods:** Random Forests e Gradient Boosting riducono l'overfitting combinando più alberi

Instabilità

Piccoli cambiamenti nei dati di training possono causare **grandi cambiamenti nella struttura dell'albero**:

- Cambiare 1-2 campioni può portare a split completamente diversi
- L'ordine dei campioni non importa, ma la composizione sì (alta varianza)

Conseguenza: il modello è difficile da fidarsi se nuovi dati sono appena diversi dal training

Soluzione: ensemble methods mitigano questo problema

Relazioni Lineari

Su pattern puramente lineari, gli alberi sono **inefficienti**:

Esempio: $y = 2x_1 + 3x_2 + 1$

Un albero dovrà creare molti split ortogonali per approssimare la retta:

```
If x1 <= 5: ...  
  If x2 <= 3: ...  
    If x1 <= 4.5: ...  
      ...
```

Mentre un modello lineare cattura la relazione con due coefficienti.

Risultato: errore più alto, overfitting per compensare

Multicollinearità

I decision tree sono **meno sensibili** della regressione lineare a feature correlate, ma non immuni:

- Tendono a **scegliere una feature** del gruppo correlato (la "prima" a ridurre impurità)
- Ignor le altre, perdendo potenzialmente informazione complementare
- Se il dataset cambia leggermente, un'altra feature potrebbe essere scelta, causando instabilità

Conseguenza: modelli potenzialmente diversi su dataset simili

Limiti di Spiegabilità

Alberi Complessi

Un albero con molti nodi (es. 100+ nodi) diventa **difficile da interpretare visualmente**:

- Non riesci più a tenere in mente l'intero modello
- Le interazioni tra feature, sebbene catturate, non sono evidenti
- È ancora tracciabile per una singola istanza, ma difficile capire il "ragionamento globale" del modello

Interazioni tra Feature

L'albero cattura le interazioni (feature A influenza l'effetto di feature B), ma **non le rende esplicite**:

- Una foglia raggiunta dopo split $[A \leq 5] \rightarrow [B > 10]$ implica un'interazione
- Ma non è ovvio dal grafico che A e B interagiscono
- Per modelli lineari, le interazioni sono esplicite se aggiunte manualmente

Bias verso Feature con Più Livelli

I decision tree **tendono a preferire feature categoriche con molti valori unici**, perché hanno più opportunità di split e quindi di ridurre impurità:

Esempio: su dataset con una feature "Città" (100 città), l'albero potrebbe scegliere molteplici split su Città, mentre feature numeriche continueranno a usare threshold.

Conseguenza: un modello che sembra dipendere dalla Città, quando in realtà potrebbe essere meno importante di un'altra feature meno "versatile".

Mitigation: limitare la profondità e usare feature importance ponderata.

Difficoltà nel Comunicare Probabilità Bassa

Se una foglia predice "Sì" ma il 60% dei campioni sono "No", la confidenza è 0.4.

Comunicare "il modello dice Sì, ma con confidenza 0.6" è meno intuitivo che dire "la probabilità stimata è 60%" (come fa la logistica).

Confronto con altri Algoritmi

Vs. Modelli Lineari (LR, Logistica)

- **Alberi:** catturano non linearità e interazioni, ma complessi e instabili
- **Lineari:** trasparenti e stabili, ma rigidi
- **Scelta:** dati con pattern complessi → alberi; dati lineari o quando interpretabilità è critica → modelli lineari

Vs. Ensemble Methods (Random Forests, Gradient Boosting)

- **Singolo albero:** interpretabile, veloce, ma prone a overfitting
- **Ensemble:** combina più alberi, miglior predizione e stabilità, ma meno interpretabile
- **Trade-off:** interpretabilità vs accuratezza
- **Quando usare:** per problemi critici dove performance conta più di interpretabilità → ensemble

Vs. SVM (Support Vector Machine)

- **Alberi:** output naturale di classe/probabilità, tracciamento facile
- **SVM:** output geometrico (margine), "scatola nera" per l'interpretazione
- **Trade-off:** SVM spesso migliore accuratezza su dati complessi, alberi più interpretabili

Vs. Neural Networks

- **Alberi:** interpretabili, nessun tuning di hyperparameter complicato
- **Neural Networks:** capacità superiore su dati ad alta dimensionalità, ma "scatola nera"
- **Trade-off:** alberi per dataset piccoli-medi e interpretabilità richiesta; NN per big data e quando l'interpretabilità non è critica

Varianti Specializzate

Alberi Potati

Rimozione iterativa di nodi per ridurre overfitting:

- **Cost-Complexity Pruning:** elimina nodi che non riducono significativamente l'errore
- **Reduced Error Pruning:** rimuove nodi usando un validation set

Extra Trees (Extremely Randomized Trees)

Simile ai Decision Tree, ma sceglie split **casualmente** invece di deterministicamente. Veloce su dataset grandi, introduce varianza che riduce overfitting su alcuni dataset.

Conditional Inference Trees

Alberi che utilizzano test statistici per la selezione di split, meno biased verso feature con più livelli.

Prompt